IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA

Project – Low Level Design

on

Cloud Cost Intelligence Platform for Finance

Student Name: Shahasad sha ali

Industry Mentor:
sumith kumar

Guided
by:sindhu

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview 3
2. System Design. 3
   2.1. Application Design 3
   2.2. Process Flow 4
   2.3. Information Flow. 5
   2.4. Components Design 5
   2.5. Key Design Considerations 5
   2.6. API Catalogue. 6
3. Data Design. 6
   3.1. Data Model 6
   3.2. Data Access Mechanism 7
   3.3. Data Retention Policies 7
   3.4. Data Migration 7
4. Interfaces 8
5. State and Session Management 8
6. Caching 8
7. Non-Functional Requirements 9
   7.1. Security Aspects 9
   7.2. Performance Aspects 9
8. References 9

1. Introduction

1.1 Scope of the document

This Low-Level Design (LLD) document provides a detailed technical specification for the AWS Cost Analytics & Reporting System. It defines the internal module design, data structures, API contracts, and implementation details required to build the system. The document serves as the primary development reference for the engineering team.

1.2 Intended Audience

Role	Purpose
Developers	Implement modules according to the specifications provided.
Technical Architects	Validate alignment with the High-Level Design.
QA Engineers	Derive test cases and integration scenarios.
DevOps	Configure deployment environments and CI/CD pipelines.

1.3 System overview

The system ingests AWS cost and usage data from CSV files, processes the information to compute service-wise and monthly cost aggregates, and presents interactive visualisations through a web dashboard. The analysis focuses on three AWS services: EC2, S3, and Lambda. The backend is built with Python/Flask and PostgreSQL; the frontend uses React and Chart.js.

2. System Design

2.1 Application Design

The system follows a three-tier architecture:

- Presentation Tier: React SPA that renders charts and tables via REST API calls.

- Application Tier: Flask API server handling ingestion, aggregation logic, and caching.
- Data Tier: PostgreSQL for persistent storage and Redis for in-memory caching.

Technology Stack Summary:

Layer	Technology
Frontend	React 18, Chart.js
Backend API	Python 3.10, Flask
Data Processing	Pandas, NumPy
Database	PostgreSQL 15
Caching	Redis

2.2 Process Flow

Ingestion and Analysis Workflow:

1. Upload: User uploads a CSV file via the dashboard.
2. Validation: API checks for required columns (DATE, SERVICE TYPE, USAGE, COST) and data types.
3. Storage: Valid records are inserted into the raw_cost_data table.
4. Aggregation: A background process computes:
    - Total cost by service (GROUP BY service_type).
    - Total cost by month (EXTRACT(MONTH FROM date)).
5. Summary Update: Aggregated results are written to service_summary and monthly_summary tables.
6. Cache Invalidation: Redis keys for aggregated data are cleared.
7. Dashboard Refresh: Frontend fetches fresh data and updates charts.

2.3 Information Flow

CSV Upload → API Validation → PostgreSQL (raw) → Aggregation Service → PostgreSQL (summary) → REST API → React Dashboard

↓

Redis Cache

- Raw data flows from ingestion to the raw_cost_data table.
- Summary data flows from aggregation queries to summary tables and Redis.
- The frontend retrieves only aggregated data for visualisation.

2.4 Components Design

Component	Responsibility	Technology
CSV Ingestion Module	Validate and parse CSV; insert records into database.	Python, Pandas
Aggregation Service	Compute service-wise and monthly cost sums; update summary tables.	SQLAlchemy, Pandas
REST API	Expose endpoints for upload, chart data, and paginated raw records.	Flask, Flask-CORS
Data Access Layer	Abstract database operations using repository pattern.	SQLAlchemy ORM
Cache Manager	Store and retrieve aggregated data to reduce database load.	Redis, Flask-Caching
Frontend Dashboard	Provide file upload interface, pie/bar charts, and data table.	React, Chart.js

2.5 Key Design Considerations

- Asynchronous Processing: Aggregation runs in a background thread (or Celery task) to keep the upload API responsive.
- Idempotency: Ingestion of the same CSV file twice does not duplicate data; duplicate detection is based on date, service_type, and usage_amount combination.
- Database Indexing: Indexes on date and service_type ensure fast aggregation queries.
- Stateless API: The Flask API does not store session data, allowing horizontal scaling.

2.6 API Catalogue

Method	Endpoint	Description	Success Response Example
POST	/ingest	Upload CSV file	{"status":"success","rows":1095}
GET	/cost/by-service	Retrieve cost totals by service	{"EC2":77800.0,"LAMBDA":23340.0,"S3":19210.0}
GET	/cost/by-month	Retrieve monthly cost totals	{"1":3797.5,"2":4480.0,...,"12":16585.0}
GET	/raw-data	Paginated raw records	{"data":[...],"total":1095,"page":1,"limit":100}
GET	/health	Health check for monitoring	{"status":"healthy"}

Error Response Format:

{"error": "Description of error", "code": 400}

3. Data Design

3.1 Data Model

Table: raw_cost_data (Raw ingested records)

Column	Type	Description
id	SERIAL PK	Auto-increment identifier
date	DATE	Usage date (e.g., 2025-01-01)
service_type	VARCHAR(50)	EC2, S3, or LAMBDA
usage_amount	DECIMAL(18,2)	Quantity consumed
cost	DECIMAL(18,2)	Cost in USD
created_at	TIMESTAMP	Ingestion timestamp

Indexes: idx_date (on date), idx_service (on service_type).

Table: service_summary (Aggregated by service)

Column	Type	Description
service_type	VARCHAR(50) PK	Service name
total_cost	DECIMAL(18,2)	Sum of all costs for the service
last_updated	TIMESTAMP	Last aggregation run

Table: monthly_summary (Aggregated by month)

Column	Type	Description
month	INTEGER PK	Month number (1–12)
year	INTEGER PK	Year (e.g., 2025)
total_cost	DECIMAL(18,2)	Sum of costs for the month
last_updated	TIMESTAMP	Last aggregation run

3.2 Data Access Mechanism

The system uses SQLAlchemy ORM with the repository pattern to isolate database operations.

Example repository method:

```
def get_service_summary():

return db.session.query(ServiceSummary).all()
```

Connection pooling is configured with a pool size of 10 to handle concurrent requests.

3.3 Data Retention Policies

- Raw data: Retained for 13 months in the database; older records are archived to AWS S3 Glacier.
- Summary tables: Retained indefinitely; they can be rebuilt from raw data if needed.

3.4 Data Migration

Schema changes are managed using Alembic migration scripts. All migrations are version-controlled and applied automatically during deployment.

4. Interfaces

The React dashboard consists of:

Screen / Component	Description
Upload Widget	File input with drag-and-drop; displays upload progress and result.
Service Pie Chart	Chart.js pie chart showing cost distribution among EC2, S3, and Lambda.
Monthly Bar Chart	Chart.js bar chart displaying cost trend across months.
Raw Data Table	Paginated table showing the most recent raw cost entries.

External Interfaces

- AWS S3 (Archive): Used for long-term storage of archived CSV files and raw data exports.
- AWS CloudWatch: Logs from the Flask API and aggregation service are streamed for monitoring.

5. State and Session Management

- Frontend: Uses React Context API for global state (chart data, upload status). No authentication tokens are currently implemented.
- Backend: The REST API is stateless. Each request contains all information needed for processing. Session data is not stored server-side.

6. Caching

Redis is used to cache aggregated chart data to improve dashboard load times.

Cache Key	Data Stored	TTL (Time to Live)
service_summary	{"EC2":..., "S3":..., ...}	1 hour
monthly_summary	{"1":..., "2":..., ...}	1 hour

Cache Invalidation: Keys are deleted immediately after a new CSV file is successfully ingested and aggregation completes.

7. Non-Functional Requirements

7.1 Security Aspects

Concern	Implementation
Input Validation	CSV file size limited to 10 MB; column names and data types strictly checked.
SQL Injection Prevention	SQLAlchemy ORM uses parameterised queries; no raw SQL concatenation.
Secure Communication	HTTPS enforced via Nginx reverse proxy in production.
File Upload Security	Uploaded files stored temporarily and deleted after processing.

7.2 Performance Aspects

Metric	Target / Implementation
API Response Time	Aggregated endpoints return in < 200ms (with cache hit).
Database Query Speed	Indexes on date and service_type keep aggregation queries under 500ms for 10^5 rows.
Concurrent Uploads	Asynchronous processing ensures upload API remains responsive under moderate load.
Pagination	Raw data endpoint returns maximum 100 records per request.

8. References

1. AWS Cost and Usage Reports – https://docs.aws.amazon.com/cur/
2. Flask Documentation – https://flask.palletsprojects.com/
3. SQLAlchemy ORM Tutorial – https://docs.sqlalchemy.org/
4. Chart.js Documentation – https://www.chartjs.org/docs/latest/
5. Redis Caching Strategies – https://redis.io/docs/manual/patterns/